Function-Oriented Roadmap

1 Data Foundation

1.1 Workload Trace Data Acquisition

1.1.1 Workload Datasets

- Collect historical workload traces for forecasting model training
- Primary datasets include:
- Google Borg cluster traces
- Alibaba cluster traces
- Download datasets in CSV or Proto formats containing time-series resource usage information
- Extract and preprocess a representative subset of the data suitable for model training and evaluation
- Optionally collect additional datasets for diversity, such as:
- BitBrains VM workload traces
- Azure VM workload traces

1.2 Anomaly Detection Dataset Acquisition

1.2.1 Labeled Time-Series Datasets

- Obtain labeled time-series datasets for anomaly detection
- Primary sources include:
- Numenta Anomaly Benchmark (NAB)

- Yahoo Service Machine Dataset (SMD)
- Parse datasets into a unified format (CSV with timestamp, metric value, and anomaly label)
- Prepare data for training and evaluation of anomaly detection models

1.3 Simulation Environment for Reinforcement Learning

1.3.1 Simulation Tool Selection

- Investigate and prototype a simulated environment for RL training, where agents interact with a controllable system rather than a static dataset
- Evaluate the following simulation approaches:
- Mininet: Network emulation with virtual hosts and switches to simulate routing decisions and resulting latencies
- CloudSim: Cloud datacenter simulator supporting virtual machines, task scheduling, and accelerated-time experiments
- Select one simulation tool for initial RL experiments
- If existing tools prove overly complex, implement a fallback custom Python simulator, for example:
- Simulate N servers with stochastic service times
- RL action: select a server
- Reward: negative request latency

1.4 Preliminary Offline Model Training

1.4.1 Forecasting Model

- Train an initial workload forecasting model using one of the collected workload trace datasets
- Example approaches include:
- ARIMA
- Prophet
- Predict future load (one hour ahead) and evaluate performance using metrics such as MAPE or RMSE

1.4.2 Anomaly Detection Model

- Train or configure a preliminary anomaly detection model using NAB or Yahoo SMD data

- Possible methods include:
- Statistical techniques (moving average with thresholds)
- Machine learning methods (Isolation Forest, LSTM autoencoder)
- Validate the models ability to detect injected or labeled anomalies

1.5 Dataset Documentation & Licensing Review

1.5.1 Data Preparation Report

- Document all datasets used, including:
- Data sources
- Preprocessing steps
- Feature selection and transformations
- Compile documentation into a Data Preparation report for inclusion in the final project documentation
- Verify dataset licensing and usage permissions, noting required attributions (Google traces under CC-BY, NAB under MIT license)

2 Telemetry & Observability Layer

2.1 Load Balancer Instrumentation

2.1.1 OpenTelemetry Integration

- Integrate the OpenTelemetry (OTel) SDK into the load balancer service
- Instrument key metrics, including:
- Request count
- Request rate
- Request latency (histograms)
- CPU and memory usage (where applicable)
- Use the OTel Go SDK (or equivalent) to ensure metrics are captured in a standardized and portable format

2.2 Metrics Storage Backend

2.2.1 Time-Series Database

- Deploy TimescaleDB as the primary time-series metrics store (PostgreSQL-based)
- Run TimescaleDB as a Docker container
- Design and create database schemas for telemetry data, such as:
- Timestamped latency measurements
- Backend/server identifiers
- Optionally deploy a Prometheus server for real-time metric scraping
- If Prometheus is used, plan for exporting long-term data to TimescaleDB via a connector

2.3 Telemetry Collection Pipeline

2.3.1 Collector Service

- Deploy an OpenTelemetry Collector service (if required)
- Configure telemetry pipelines with the following stages:

- Input: OTLP or Prometheus scraping
- Processing: batching and basic transformations
- Output: TimescaleDB, Prometheus remote write, or file-based export
- Ensure reliable delivery of metrics from the load balancer to storage

2.4 Distributed Tracing

2.4.1 Tracing Setup

- Initialize OpenTelemetry tracing in the load balancer service
- Optionally instrument dummy backend services for end-to-end tracing
- Configure trace output to logs or a tracing backend (Jaeger)
- Treat this task as optional if sprint time becomes constrained

2.5 Metrics Visualization

2.5.1 Grafana Dashboard

- Deploy a lightweight visualization tool (Grafana)
- Connect the dashboard to TimescaleDB (or Prometheus, if used)
- Create an initial dashboard displaying:
- Request rates over time
- Latency distributions
- Use visualization primarily to validate correct data ingestion

2.6 Telemetry Data Validation

2.6.1 Synthetic Load Testing

- Generate synthetic load using tools such as curl or simple scripts
- Verify that telemetry data appears correctly in the metrics database
- Validate:
- Metric resolution and accuracy
- Correct labeling (server IDs, request paths)
- Refine instrumentation as needed to ensure data usability for ML modules

3 Core Routing & Infrastructure

3.1 Repository & Version Control Setup

3.1.1 Project Repository Initialization

- Initialize the SmartLoad GitHub repository (public or private as appropriate)
- Define project directory structure for each microservice/module:
- load-balancer/
- rl-engine/
- telemetry/
- Add a clear README describing the project and structure
- Establish branching and merging conventions (feature branches, pull requests)
- Ensure repository access for all team members

3.2 CI/CD Pipeline Configuration

3.2.1 Continuous Integration

- Set up continuous integration using GitHub Actions (or equivalent)
- Configure workflows to:
- Lint code on each push
- Run a basic (initial) test suite
- Build Docker images
- Plan for future expansion of the pipeline to include module-level testing

3.3 Development Environment & Containerization

3.3.1 Docker-Based Setup

- Create Dockerfiles for each major component
- Provide a docker-compose.yml to orchestrate services in development
- Initially support a minimal setup (placeholder services)

- Verify that a simple Hello World service (load balancer ping response) runs successfully via Docker

3.4 Baseline Architecture Finalization

3.4.1 Technology Stack Decisions

- Review and confirm technology stack decisions (detailed in the next section)
- Agree on:
- Programming languages and frameworks per component
- Responsibilities of each module
- Walk through the Modular Architecture Overview
- Define and document interfaces between components, including:
- Control bus data
- Telemetry message formats

3.5 Sprint Planning & Task Tracking

3.5.1 Project Management Setup

- Set up GitHub Projects or Issues for task management
- Create and populate Sprint 1 tasks
- Define clear acceptance criteria (CI passing, repo structure finalized)

3.6 Load Balancer Service Implementation

3.6.1 HTTP Reverse Proxy

- Develop the traffic ingress component that accepts client HTTP requests and forwards them to backend servers
- Implement the service as a minimal HTTP reverse proxy:
- Primary choice: Go using the built-in net/http library
- Fallback option: Node.js with Express (if Go learning is delayed)
- Implement Round-Robin routing as the initial load balancing policy
- Configure the service to read backend server addresses from:
- Configuration files, or
- Environment variables

3.7 Dummy Backend Services

3.7.1 Backend Pool

- Implement a lightweight backend service (Python/Flask or Node.js)
- Return a fixed response to incoming requests
- Optionally simulate processing latency (sleep for a few milliseconds)
- Run multiple instances in separate containers to form the backend server pool

3.8 Baseline Health Check Mechanism

3.8.1 Backend Monitoring

- Implement periodic health checks from the load balancer to the backend instances
- Detect unresponsive backends and exclude them from request routing
- Maintain simple round-robin behavior over only healthy instances
- Lay the groundwork future anomaly detection integration

3.9 Logging & Basic Telemetry Collection

3.9.1 Request-Level Logs

- Log each incoming request handled by the load balancer
- Capture, at minimum:
- Timestamp
- Selected backend instance
- Response time
- Output telemetry to standard output or log files as an initial baseline
- Prepare for later integration with the full telemetry pipeline

3.10 Unit Testing of Load Balancer Logic

3.10.1 Routing Tests

- Write unit tests for core routing behavior, including:
- Correct round-robin server selection
- Proper handling of backend removal or failure

- Integrate tests into the CI pipeline to ensure reliability

3.11 Code Review & Integration

3.11.1 Peer Review Process

- Conduct peer code reviews prior to merging into the main branch
- Ensure adherence to quality, correctness, and shared understanding

4 Decision Intelligence Layer

4.1 Anomaly Detection Microservice

4.1.1 Detection Engine

- Implement the anomaly detection engine as a standalone microservice
- Use Python to leverage established ML libraries (scikit-learn or PyTorch)
- Consume telemetry data by:
- Subscribing to the telemetry stream, or
- Periodically querying the metrics database
- Start with a simple statistical baseline, such as:
- Rolling averages and standard deviations
- Threshold-based deviation detection for latency or error rate
- Integrate a more advanced anomaly detection model (Isolation Forest or LSTM-based model) for improved accuracy

4.2 Real-Time Anomaly Signaling

4.2.1 Control Bus Integration

- Define how anomaly alerts are published within the Decision Layer
- Emit per-node health status signals indicating normal, degraded, or unhealthy states
- Implement signaling using one of the following approaches:
- Preferred: lightweight pub/sub mechanism (Redis Pub/Sub or MQTT)
- Fallback: status updates stored in the metrics database or an in-memory store
- Favor asynchronous, event-driven communication for scalability and decoupling

4.3 Load Balancer Integration & Routing Constraints

4.3.1 Failure-Aware Routing

- Modify the load balancer to consume anomaly signals via:

- Subscription to the control bus, or
- Polling the anomaly detection service API
- Enforce routing rules such that:
- Instances flagged as unhealthy are excluded from routing
- Degraded instances are down-weighted in server selection
- Ensure that anomaly-based constraints are applied prior to other routing policies
- Define recovery logic for backend instances, such as:
- Automatic reintegration once anomalies clear, or
- Manual reset for safety

4.4 Anomaly Scenario Testing

4.4.1 Validation Scenarios

- Simulate abnormal backend behavior, including:
- Artificial slowdowns
- Increased error rates or unresponsiveness
- Verify that the anomaly detection service correctly flags problematic instances
- Confirm that the load balancer avoids or deprioritizes flagged nodes
- Replay labeled anomaly sequences from NAB or Yahoo SMD datasets as pseudo-live telemetry input
- Measure end-to-end detection and mitigation latency, targeting response within one or two monitoring intervals

4.5 Forecasting Microservice

4.5.1 Workload Prediction Engine

- Implement the workload forecasting engine as a standalone microservice
- Use Python to leverage time-series forecasting libraries (Prophet, statsmodels)
- Periodically query the metrics database for historical load data, such as:
- Request rates
- CPU utilization
- Generate short-term load predictions (next 5-minute or 1-hour interval)

- Begin with a simple baseline model (last-hour average predicts next-hour load) to validate data flow
- Integrate a more advanced forecasting model trained (ARIMA, Prophet, or LSTM) once the pipeline is verified
- Output predicted load values or ranges via the control bus

4.6 Autoscaling Decision Logic

4.6.1 Autoscaler Module

- Design and implement the Autoscaler as a separate module for clarity and modularity
- Subscribe to forecast outputs published on the control bus
- Use forecasted load (and optional auxiliary signals) to decide scaling actions
- Implement initial rule-based logic, for example:
- Scale up if predicted load exceeds current capacity by X%
- Scale down if predicted load remains below capacity by Y% for Z minutes
- Encode conservative defaults to avoid oscillations or thrashing

4.7 Scaling Infrastructure Integration

4.7.1 Instance Management

- Implement mechanisms to execute scaling actions at the infrastructure level
- For a local or simulated environment:
- Launch new dummy backend instances as Docker containers
- Terminate containers to simulate scale-down
- Maintain a registry of active backend instances, such as:
- A configuration file
- A database table
- Ensure the load balancer dynamically updates its backend pool when the registry changes
- If a cloud testbed is available, optionally integrate with real infrastructure APIs (AWS EC2 via Boto3) to manage instances

4.8 End-to-End Control Loop Integration

4.8.1 Control Loop Validation

- Validate the full control loop:
- Telemetry Forecast Autoscale New Resources Telemetry
- Simulate rising load by increasing request rates to the load balancer using a load generation script
- Verify that:
- The forecasting service predicts the upward trend
- The autoscaler proactively provisions additional instances
- The load balancer routes traffic to newly added instances
- Similarly, test sustained low-load scenarios to validate scale-down behavior

4.9 Autoscaling Safety Constraints

4.9.1 Scaling Policies

- Introduce basic policy constraints to prevent unsafe scaling decisions
- Enforce:
- Maximum number of backend instances (cost control)
- Minimum number of instances (fault tolerance)
- Hard-code constraints initially or load them from configuration files
- Treat these as a precursor to the full Policy Manager

4.10 Reinforcement Learning Decision Engine

4.10.1 RL Routing Module

- Implement the RL-based routing engine as a standalone module
- Use Python with an RL framework (Stable Baselines3 with PyTorch)
- Define the RL formulation:
- State: current load, per-server latency, server health flags from the anomaly detector
- Action: select a backend server or output a ranked distribution of servers
- Reward: encourage low latency and balanced load (negative P95 latency, throughputlatency trade-off)

- Start with a simple RL algorithm to validate end-to-end learning, such as:
- Policy gradient (REINFORCE), or
- Q-learning in a discrete action space

4.11 Offline Training in Simulation

4.11.1 RL Training

- Train the RL agent offline using the simulation environment developed
- Use CloudSim or a custom simulator to run many training episodes without affecting the live system
- Simulate long-duration workloads in accelerated time
- Monitor training progress using reward curves and policy behavior
- Verify convergence toward reasonable routing behavior (favoring faster or less loaded servers)
- Save trained model parameters for deployment

4.12 Shadow Deployment of RL Engine

4.12.1 Shadow Mode

- Integrate the RL engine into the live SmartLoad system in shadow mode
- Allow the RL engine to:
- Consume live telemetry
- Compute routing recommendations
- Log RL decisions without enforcing them, while the load balancer continues using classical routing
- Compare RL-recommended actions with actual routing decisions to validate correctness and safety
- Collect performance data under test loads or controlled scenarios

4.13 Full RL Integration with Routing Logic

4.13.1 Decision Hierarchy

- Enable RL-driven routing decisions in the load balancer once sufficient confidence is established

- Implement a clear decision hierarchy:
- (1) Apply anomaly detection constraints to remove unhealthy nodes
- (2) Use RL-generated routing scores or rankings
- (3) Fall back to classical load balancing if RL output is unavailable, uncertain, or warming up
- Allow policy overrides (safe mode) to bypass RL decisions when required
- Implement RL communication via the control bus, for example:
- Publish server rankings or traffic distributions
- Load balancer consumes RL messages and routes accordingly

4.14 Online Learning Strategy & Safety Controls

4.14.1 Learning Policies

- Decide between:
- Fixed-policy deployment trained offline, or
- Controlled online learning with continual updates
- If online learning is enabled:
- Limit exploration to avoid unsafe routing behavior
- Gate exploration parameters through configuration or policy constraints
- Favor conservative defaults suitable for short-term project safety

4.15 Evaluation of RL-Based Routing

4.15.1 RL Experiments

- Design experiments to evaluate RL effectiveness, including:
- Scenarios with one consistently slower backend server
- Time-varying workloads and traffic spikes
- Compare RL-based routing against classical algorithms by measuring:
- Average latency
- P95 latency
- Load distribution fairness
- Use telemetry data to quantitatively assess performance gains

5 Policy & System Integration

5.1 Policy Manager Implementation

5.1.1 Policy Configuration

- Develop a lightweight Policy Manager to centralize configuration and high-level control
- Implement policy distribution using one of the following:
- Shared JSON/YAML configuration files, or
- A small configuration service exposing a policy API
- Support core policy categories, including:
- Operating Mode: classical-only, hybrid (RL + classical), or learning-only
- Safe Mode: force classical routing and disable RL outputs
- Scaling Limits: minimum and maximum number of instances
- SLA Targets: desired maximum P95 latency
- RL Parameters: exploration rate, reward toggles, etc.
- Ensure all SmartLoad components can read updated policies via the control bus or a shared configuration store

5.2 Security & Configuration Validation

5.2.1 Secrets & Validation

- Ensure sensitive configuration values (cloud API keys) are not hard-coded
- Store secrets securely using environment variables
- Add basic validation to detect invalid or unsafe policy values

5.3 End-to-End Integration Testing

5.3.1 Integration Scenarios

- Conduct comprehensive integration tests across all system components
- Validate the following end-to-end scenarios:

- Scenario 1: steady load node anomaly traffic rerouted recovery and reintegration
- Scenario 2: sudden load spike forecast predicts increase autoscaler provisions new instance RL routes traffic latency improves
- Test scale-down behavior after load decreases and verify grace periods

5.4 Failure Handling & Fallback Validation

5.4.1 Failure Scenarios

- Simulate partial system failures, including:
- RL engine crash or unavailability
- Metrics database outage
- Verify that the system continues operating safely using classical load balancing and default behavior
- Ensure components degrade gracefully using cached or last-known data

5.5 Resource Usage Monitoring & Optimization

5.5.1 Performance Monitoring

- Monitor CPU and memory usage of each SmartLoad component under load
- Identify and optimize any performance bottlenecks
- Ensure the overall system remains lightweight and efficient

5.6 Performance Tuning & Parameter Refinement

5.6.1 Parameter Optimization

- Tune anomaly detection thresholds to balance sensitivity and stability
- Refine forecasting frequency and model parameters for accuracy and efficiency
- Adjust RL hyperparameters (learning rate, exploration) and retrain models if improvements are observed
- Optimize the load balancer implementation for throughput (Go concurrency or Node.js clustering)

5.7 Optional UI / Dashboard Integration

5.7.1 Operator Interface

- Implement a simple web UI or CLI for operators if time permits
- Display real-time system status, including:
- Current load
- Active backend instances
- Detected anomalies
- Allow toggling key policies (enabling or disabling safe mode)
- If implementation time is limited, prepare static dashboards or logs (Grafana) for demonstration purposes

6 Testing & Benchmarking

6.1 Automated Test Suite Development

6.1.1 Testing Framework

- Develop a comprehensive automated testing framework covering unit, integration, and regression tests
- Integrate tests into the CI pipeline to ensure repeatability and long-term reliability

6.2 Unit Testing of Individual Modules

6.2.1 Module Tests

- Complete unit tests for any remaining components, including:
- Anomaly detection logic (flagging known anomalous sequences)
- Forecasting module outputs on known patterns
- RL agent decision logic on mock system states
- Load balancer routing logic and fallbacks
- Responsibility split by module ownership:
- ML-related modules (forecasting, anomaly detection, RL)
- load balancer and routing logic

6.3 End-to-End Integration Testing

6.3.1 System Scenarios

- Develop integration test scripts that bring up the full SmartLoad system (via Docker Compose)
- Simulate realistic scenarios such as:
- Gradually increasing request load
- Autoscaler-triggered instance provisioning
- RL-guided routing under varying load

- Validate outcomes using assertions, for example:
- Number of active backend instances increases as expected
- P95 latency remains below configured SLA targets
- Implement tests using Python scripts or a testing framework

6.4 Regression Testing

6.4.1 Bug Prevention

- Identify bugs or edge cases discovered during integration
- Add targeted regression tests to prevent recurrence

6.5 Performance Benchmarking

6.5.1 Load Testing

- Use a load generation tool (Locust, JMeter, or a custom multithreaded script) to stress the system
- Gradually increase load until system capacity is reached
- Measure key performance indicators, including:
- Throughput (requests per second)
- Latency distribution (average and P95 latency)
- Benchmark alternative configurations, such as:
- RL-based routing vs classical round-robin routing
- Forecasting-based autoscaling vs reactive autoscaling
- Measure SmartLoads own resource consumption (CPU and memory usage per microservice) to assess system overhead

6.6 Model Accuracy Evaluation

6.6.1 ML Metrics

- Evaluate forecasting accuracy by comparing predicted versus actual load trends
- Assess anomaly detection performance using labeled datasets or known injected anomalies
- Report false positive and false negative rates where applicable

6.7 Fault Tolerance & Resilience Testing

6.7.1 Failure Injection

- Intentionally disrupt system components to validate graceful degradation, including:
- Terminating the RL engine and verifying fallback to classical routing
- Disabling the forecasting service and confirming autoscaling continues in reactive mode
- Ensure no system-wide crashes occur and default behavior maintains service availability

6.8 System Tuning Based on Results

6.8.1 Post-Benchmark Adjustments

- Analyze benchmarking results against SLA targets
- If P95 latency exceeds acceptable bounds:
- Adjust autoscaling thresholds or instance limits
- Optimize load balancer code paths
- Refine ML components as needed, such as:
- Retraining RL models with additional data
- Adjusting reward functions or forecasting parameters
- Ensure compliance with any explicit performance or efficiency requirements specified by supervisors

6.9 Results Documentation & Reporting

6.9.1 Experimental Artifacts

- Collect logs, metrics, and experiment outputs
- Generate plots and charts for key results, such as:
- Latency vs load curves
- Scaling behavior over time
- Tabulate KPIs comparing baseline behavior with SmartLoad-enabled behavior for inclusion in the final report

7 Deployment & Finalization

7.1 Production / Demo Deployment Setup

7.1.1 Deployment Environment

- Prepare a realistic deployment environment for demonstration
- Deploy SmartLoad using one of the following approaches:
- Cloud-based virtual machines (AWS, Azure, or university cluster)
- Kubernetes cluster with SmartLoad components as services
- Assign roles within the deployment:
- Load balancer deployed as a public-facing service
- Backend services deployed as separate instances or pods
- Configure networking to ensure:
- Load balancer endpoint is externally accessible
- Internal communication with backend services is functional
- Harden deployment artifacts:
- Production-grade Docker Compose configuration, or
- Helm charts if using Kubernetes
- Ensure all environment variables (database connections, service endpoints) are correctly parameterized

7.2 Final Code Cleanup & Refactoring

7.2.1 Code Quality

- Refactor code for clarity and maintainability
- Remove or disable debug-only logging
- Ensure all configuration values are externally configurable
- Eliminate hard-coded constants and magic values

7.3 Documentation Completion

7.3.1 Project Documentation

- Consolidate documentation produced across all sprints
- Ensure consistency between design documents and final implementation

7.4 Sprint Reports Compilation

7.4.1 Development Narrative

- Merge sprint-level notes into a cohesive development narrative
- For each sprint, summarize:
- Objectives
- Key implementation outcomes
- Deviations from the original plan

7.5 User Guide Preparation

7.5.1 User Instructions

- Write a concise guide explaining how to:
- Deploy and start SmartLoad services
- Reproduce key test or demo scenarios
- Interpret logs, metrics, and outputs
- Include the guide in the repository README or as a standalone markdown document

7.6 Technical Documentation Finalization

7.6.1 Module Documentation

- Ensure each SmartLoad module includes:
- A brief functional description
- Interface definitions
- Key design decisions
- Update existing module design documents (Anomaly Detection, Decision Layer) to reflect the final system state

7.7 Final Project Report Writing

7.7.1 Academic Report

- Prepare the final written report following the expected academic structure:
- Introduction and problem statement
- Related work
- System architecture and design rationale
- Implementation details
- Testing and performance evaluation
- Conclusions and future work
- Divide drafting responsibilities:
- introduction and architecture
- AI/ML components and experimental results
- infrastructure, deployment, and testing
- Collaboratively edit and refine the report for coherence and quality

7.8 Presentation & Demo Preparation

7.8.1 Final Demonstration

- Prepare presentation slides summarizing:
- Motivation and system overview
- Key technical contributions
- Experimental results and insights
- Design a live demo or scripted demonstration showcasing:
- Load scaling behavior
- Anomaly handling
- RL-based adaptive routing
- Rehearse the presentation and demo to ensure smooth execution

7.9 Final Review & Buffer Time

7.9.1 Final Checks

- Reserve time for last-minute fixes or refinements

- Address feedback from supervisors, if any
- Verify that all deliverables meet submission requirements

8 Parallel Team Streams

8.1 Stream A: Data & Decision Intelligence

- Data Foundation
- Decision Intelligence Layer

8.2 Stream B: Infrastructure & Observability

- Core Routing & Infrastructure
- Telemetry & Observability Layer

8.3 Stream C: Integration, Testing, and Deployment

- Policy & System Integration
- Testing & Benchmarking
- Deployment & Finalization